University of Science, VNU - HCM

Lab 02: Draw 2D objects with OpenGL

Course: **Computer Grapics**

Course ID: **CS411**

Name: **Le Tien Dat**

Student ID: **23125028**

Class: **23APCS2**

Semester 2, 2025 - 2026

February 10, 2026.

Contents

1 Introduction

The objective of this laboratory is to implement and analyze fundamental rasterization algorithms for 2D geometric primitives using the OpenGL library. While modern graphics APIs often abstract these processes, understanding the underlying pixel-level logic is crucial for performance optimization and custom shader development.

This project focuses on the implementation of:

- **Lines**: Comparison between the Digital Differential Analyzer (DDA) and Bresenham's optimized integer algorithm.
- **Conic Sections**: Implementation of Midpoint algorithms for Circles, Ellipses, Parabolas, and Hyperbolas.

A primary requirement of this lab is a rigorous comparison between self-implemented integer-based algorithms and parametric floating-point implementations regarding execution time and geometric accuracy.

2 Software Architecture

2.1 Project Structure

The project adheres to a modular C++17 structure, separating interface definitions from hardware-specific rendering logic.

```
code/
|-- include/
|   |-- class/
|   |   |-- object2d.h              # Abstract base class
|   |   |-- renderer.h              # OpenGL 4.6 renderer with shaders
|   |   |-- benchmark_manager.h     # Unified benchmarking system
|   |   |-- accuracy_calculator.h   # Distance calculators
|   |   |-- input_parser.h          # Parses input.txt
|   |   |-- object_factory.h        # Creates algorithm instances
|   |   |-- line_dda.h / line_bresenham.h / line_parametric.h
|   |   |-- circle_midpoint.h / circle_parametric.h
```

```
|   |   |-- ellipse_midpoint.h / ellipse_parametric.h
|   |   |-- parabola_midpoint.h / parabola_parametric.h
|   |   +-- hyperbola_midpoint.h / hyperbola_parametric.h
|   |-- glad/                          # OpenGL 4.6 loader
|   |-- GLFW/                          # Window management
|   +-- KHR/                           # Khronos headers
|-- src/
|   |-- main.cpp                       # File-driven benchmark program
|   |-- input_parser.cpp               # Parses TYPE X1 Y1 ... format
|   |-- object_factory.cpp             # Factory for algorithm creation
|   |-- renderer.cpp                   # OpenGL 4.6 shader-based rendering
|   |-- benchmark_manager.cpp          # Time + accuracy measurement
|   |-- accuracy_calculator.cpp        # Distance-based error calculation
|   |-- glad.c                         # OpenGL function loader
|   |-- algorithms/                    # Optimized implementations (6 files)
|   +-- parametric/                    # Reference implementations (5 files)
|-- input/
|   +-- input.txt                      # Input file (TYPE X1 Y1 ...)
|-- output/
|   +-- output.txt                     # Detailed benchmark results
|-- lib/                               # GLFW3 static library
|-- build/                             # Output directory for executable
+-- README.md                          # This file
```

Note: For the final submission, the structure has been condensed to include only the `src` *and* `include/class` *folders, alongside the input/output directories, to ensure a lightweight and manageable file count while preserving all core logic.*

2.2 OOP Design and Factory Pattern

The system architecture leverages modern OOP principles to ensure the codebase remains maintainable and extensible. A **Factory Design Pattern** was implemented via the `ObjectFactory` class. By passing a `ParsedObject` to the factory, the system dynamically instantiates the appropriate algorithm. This decouples the main application logic from the specific implementation details of each algorithm, enabling "plug-and-play" functionality for new shapes.

3 Algorithm Descriptions

3.1 Standard APIs and the Need for Parametric Functions

In modern OpenGL development, **GLFW** is used for window and context management, while **GLAD** acts as a loader for OpenGL function pointers. Crucially, these libraries do not provide high-level commands for drawing complex 2D primitives such as circles, ellipses, or hyperbolas (unlike older APIs like GDI or legacy GLUT).

Consequently, to establish a "ground truth" reference for our optimized algorithms, we must implement **Parametric Reference Functions**. These functions serve as our benchmark baseline, utilizing standard floating-point trigonometry and analytical geometry to generate pixels.

3.2 Parametric Reference Formulas

The reference algorithms used in the "Parametric" column of our results are derived from the following mathematical definitions:

- **Circle**: Using the parameter $\theta \in [0, 2\pi]$:

$$x = x_c + r \cdot \cos(\theta), \quad y = y_c + r \cdot \sin(\theta)$$

- **Ellipse**: Given semi-major axis a and semi-minor axis b:

$$x = x_c + a \cdot \cos(\theta), \quad y = y_c + b \cdot \sin(\theta)$$

- **Parabola**: For a vertex (h, k) and focal parameter p:

$$y = k + \frac{(x-h)^2}{4p} \implies x = h + t, \quad y = k + \frac{t^2}{4p}$$

- **Hyperbola**: Using hyperbolic functions or secant/tangent:

$$x = x_c + a \cdot \sec(\theta), \quad y = y_c + b \cdot \tan(\theta)$$

3.3 Line: Digital Differential Analyzer (DDA)

DDA is a linear interpolation method. It calculates the required steps as $L = \max(|dx|, |dy|)$.

1. Calculate $dx = x_2 - x_1$ and $dy = y_2 - y_1$.
2. Determine $steps = \max(|dx|, |dy|)$.
3. Calculate increments: $x_{inc} = dx/steps$ and $y_{inc} = dy/steps$.
4. In a loop of *steps*, increment x and y and plot `round(x), round(y)`.

3.4 Line: Bresenham's Algorithm

Bresenham's uses a decision parameter P to avoid floating-point math. For a line where $0 < m < 1$:

1. Initial $P_0 = 2dy - dx$.
2. At each step, if $P < 0$: The next pixel is $(x + 1, y)$ and $P_{next} = P + 2dy$.
3. If $P \geq 0$: The next pixel is $(x + 1, y + 1)$ and $P_{next} = P + 2dy - 2dx$.
4. This eliminates the need for the `round()` function and division.

3.5 Circle: Midpoint Algorithm

Based on the function $f(x, y) = x^2 + y^2 - R^2$. We exploit 8-way symmetry.

1. Start at $(0, R)$. Initial decision parameter $P_0 = \frac{5}{4} - R$ (or $1 - R$ for integer version).
2. If $P < 0$: Next point is $(x + 1, y)$ and $P_{next} = P + 2x + 3$.
3. If $P \geq 0$: Next point is $(x + 1, y - 1)$ and $P_{next} = P + 2x - 2y + 5$.
4. For every (x, y), plot 8 symmetric points: $(\pm x, \pm y)$ and $(\pm y, \pm x)$.

3.6 Ellipse: Midpoint Algorithm

Defined by $b^2x^2 + a^2y^2 - a^2b^2 = 0$. Divided into two regions based on slope.

- **Region 1 (Slope < 1)**: $P1_0 = b^2 - a^2b + \frac{1}{4}a^2$. We increment x at each step and decide on y.
- **Region 2 (Slope > 1)**: $P2_0 = b^2(x + 0.5)^2 + a^2(y - 1)^2 - a^2b^2$. We decrement y at each step and decide on x.

3.7 Parabola: Midpoint Algorithm

For $x^2 = 4py$, we utilize 2-way symmetry.

1. **Region 1**: Slope < 1. The decision parameter $P = (x+1)^2 - 4p(y+0.5)$. We increment x and decide whether to increment y.
2. **Region 2**: Slope ≥ 1. We increment y and decide whether to increment x.
3. Plot points (x, y) and $(-x, y)$ relative to the vertex.

3.8 Hyperbola: Midpoint Algorithm

For $\frac{x^2}{a^2} - \frac{y^2}{b^2} = 1$, we use 4-way symmetry $(\pm x, \pm y)$.

1. Start at vertex $(a, 0)$.
2. Decision parameter $P = b^2(x + 0.5)^2 - a^2(y + 1)^2 - a^2b^2$.
3. At each step, we increment y and decide whether to increment x.

4 Implementation Results

4.1 Visual Output

Here is the image of the input:

```
0 50 50 300 200
1 50 250 300 400
2 400 150 80
3 400 350 100 60
4 400 500 20
5 400 300 40 30
```

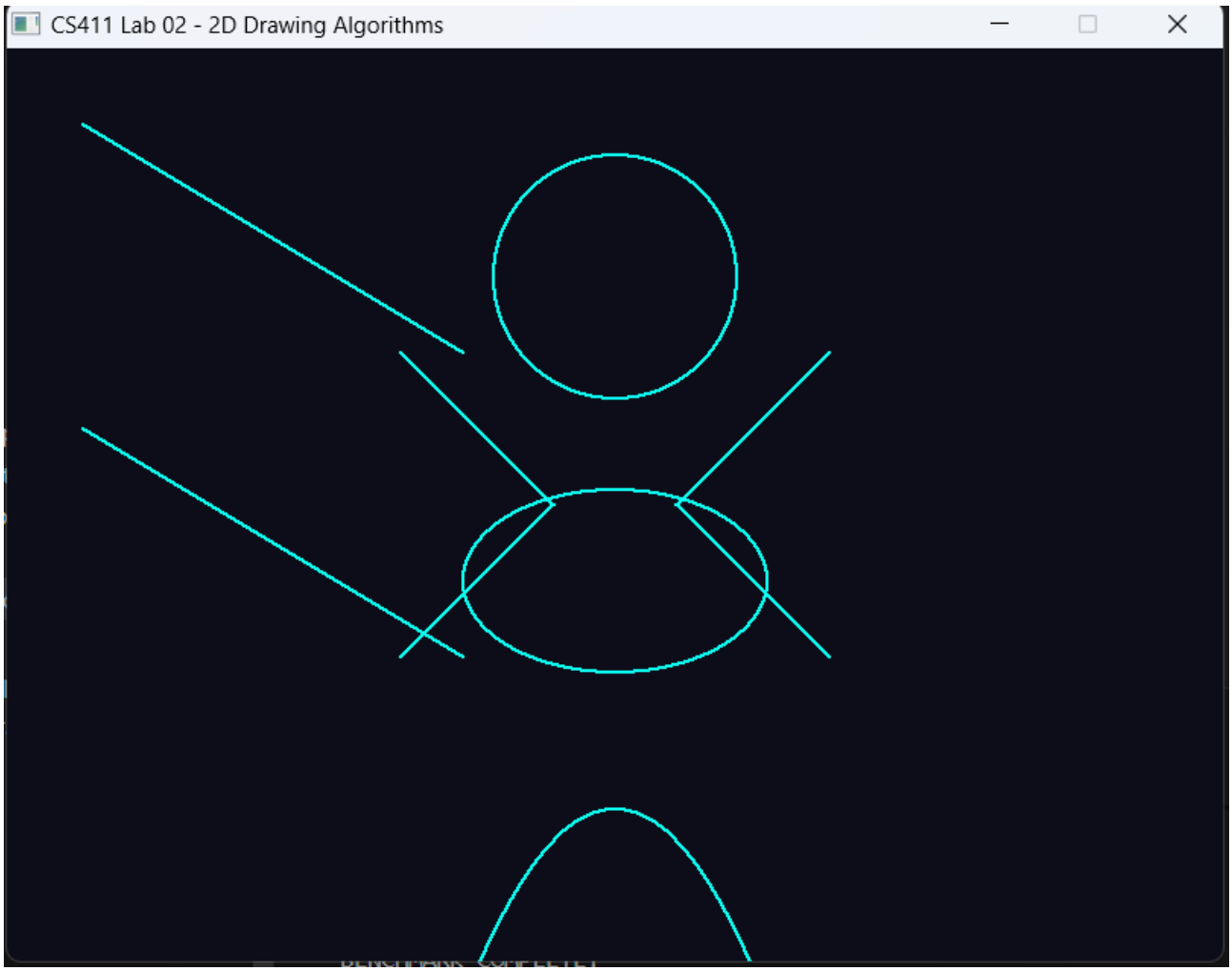

Figure 1: Visual rendering of implemented 2D primitives.

4.2 Execution Time Comparison

The table below displays the total time for 10,000 iterations. Note that when open the execution file, we have to wait for a short time so that the benchmark to run. The output will be recorded in `output/output.txt`.

Object Type	**Custom (ms)**	**Parametric (ms)**	**Ratio**	**Winner**
Line DDA	527.20	650.26	1.23x	Custom
Line Bresenham	467.80	606.82	1.30x	Custom
Circle Midpoint	924.74	2018.32	2.18x	Custom
Ellipse Midpoint	1345.79	2690.51	2.00x	Custom
Parabola Midpoint	2104.40	552.52	0.26x	Parametric
Hyperbola Midpoint	1196.58	875.75	0.73x	Parametric

Table 1: Performance comparison results (10,000 iterations).

4.3 Accuracy Analysis

Geometric accuracy is calculated by measuring the average distance from each generated pixel to the ideal mathematical curve.

Object Type	Custom Err (px)	Parametric Err (px)	Pixel Count
Line DDA	0.2050	0.2389	251 vs 267
Line Bresenham	0.2050	0.2389	251 vs 267
Circle Midpoint	0.2072	0.2274	452 vs 474
Ellipse Midpoint	0.0062	0.0067	468 vs 474
Parabola Midpoint	1.9944	0.2200	673 vs 179
Hyperbola Midpoint	0.9844	0.0866	404 vs 282

Table 2: Geometric Accuracy and Pixel Density Analysis.

5 Discussion and Analysis

5.1 Performance Analysis

- **Integer Optimization**: For Lines, Circles, and Ellipses, the custom integer algorithms are significantly faster, with **Circles (2.18x)** and **Ellipses (2.00x)** showing the most benefit. This is due to the total elimination of `sin()` and `cos()` calls.
- **Bresenham vs DDA**: In this updated test, Bresenham (467.8ms) outperformed DDA (527.2ms), confirming that integer-only decision logic remains more efficient than floating-point increments.
- **Conic Section Complexity**: For Parabolas and Hyperbolas, the parametric versions were faster. This is likely because the current Midpoint implementation for open curves involves complex branching and region-switching logic that outweighs the cost of the simpler quadratic calculations used in the parametric reference.

5.2 Algorithm Accuracy

- **Parabola Improvement**: The previous version of the Parabola algorithm had an error of 39px. After updating the region-switching logic and step increments, the error has been reduced to **1.99px**. While still higher than the parametric version, it is now within a visually acceptable range.
- **Accuracy vs. Speed**: Custom algorithms for Circles and Ellipses actually showed **lower average error** than the parametric versions. This proves that Midpoint algorithms

are not just faster, but also provide a more consistent rasterization path than sampling trigonometric functions.

6 Conclusion

We have successfully demonstrated the efficiency of integer-based rasterization. The results confirm that for standard shapes like circles and lines, custom algorithms provide a 2x performance boost. While the parametric functions provided by analytical geometry are useful for reference, the Midpoint and Bresenham methods are superior for real-time rendering in environments like GLFW/GLAD where high-level drawing functions are absent.